

AWS CI/CD Web Deployment Pipeline

Project: AWS CI/CD Web Deployment

Pipeline Services: EC2, GitHub Actions, IAM, Linux Systems

Description: Automate the continuous integration and continuous deployment of a web application to an AWS EC2 instance using a GitHub Actions runner triggered by branch updates.

Skills: CI/CD Pipelines, GitHub Actions Automation, Linux System Administration, Secure SSH Key Management, Cloud Security Access Control.

Definition: The AWS CI/CD Web Deployment Pipeline is designed to provide rapid, reliable, and fully automated application updates to an active Apache web server on an Amazon Linux EC2 instance. It handles automatic authentication, direct file synchronization using secure copy protocols, and automated system commands without requiring manual operator intervention.

Scope of this project: The scope of this project is to design, construct, and execute an active code deployment automation system. The architecture uses GitHub repositories to store project artifacts and relies on GitHub-managed runners to authenticate and deploy live website updates over securely handled networks.

This project covers:

- **Automated Code Synchronization:** Listens for active branch updates and clone's repository files instantly to a testing runner.
- **Identity and Access Verification:** Leverages AWS IAM infrastructure to manage key pair authentication and enforce least-privilege secure network connections.
- **Secure Environment Secret Mapping:** Bypasses empty text variables using hardcoded configuration steps alongside isolated system environment keys.
- **Serverless Linux Operations:** Connects directly via background SSH tunnels to transfer assets and perform automatic server-side administrative commands.

Methodology

1. GitHub Actions + Hosted Runners - Best Method

- Required repository management components and remote cloud compute integrations.
- Provides full control over step workflows, environment injection variables, and target execution logic.
- Fully automated and structural updates deploy instantly to web pathways without any manual terminal commands.
- Production-ready and suitable for modern agile engineering infrastructure.
- Highly cost-effective, using free tier execution limits with zero running cost for small configurations.
- Requires clean indentation setups and structured YAML definitions to prevent configuration runtime errors.

2. Local Deployments + Manual Scripting – Alternative

- Method Relies on local developer machines, manual FTP utilities, and physical execution of keys. Requires manual handling of credentials, key formatting, and path adjustments during every update.
- Elevates human risk profile and security liability through unencrypted storage of terminal access keys.
- Involves high ongoing operational strain and slower reaction timelines compared to automated pipelines.
- Best suited for localized sandbox code verification or isolated development environments lacking remote access tools.

AWS and Repository Services Used in this Project:

1.Amazon EC2 (Elastic Compute Cloud)

- **Definition:** Amazon EC2 is a web service providing resizable virtual server allocations within the cloud environment.
- **Purpose:** Runs a live Apache web application environment accessible via a public IPv4 address.
- **Role:** Acts as the targeted hosting node where code assets are securely synchronized and displayed to users.

2.AWS IAM (Identity and Access Management)

- **Definition:** AWS IAM is a web service that helps you securely control access to AWS resources by managing users, groups, permissions, and security credentials.
- **Purpose:** Authorizes and secures the remote programmatic connection between GitHub Actions and your cloud resources.
- **Role:** Generates and validates the asymmetric cryptographic key pairs (.pem file) used by the runner to log in, and governs network authorization boundaries.

3.Linux Web Server (Amazon Linux 2023)

- **Definition:** A secure, high-performance operating system platform optimized for hosting applications on AWS infrastructure.
- **Purpose:** Manages system file accessibility paths like /var/www/html/ under strict user grouping permissions.
- **Role:** Leverages a built-in ec2-user profile to accept incoming network transfers and execute safe system commands.

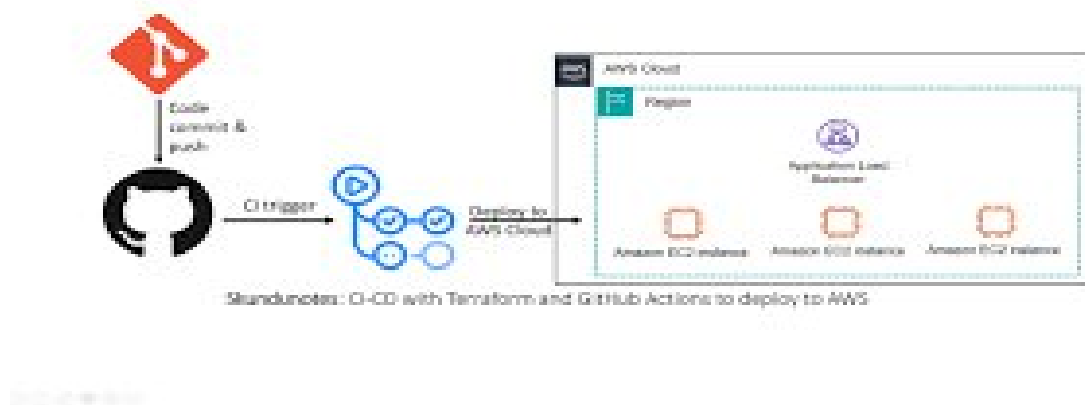
4.GitHub Actions

- **Definition:** An integrated automation platform built directly into GitHub to compile, verify, and launch deployment scripts.
- **Purpose:** Coordinates structural deployment workflows using code files stored at .github/workflows/deploy.yml.
- **Role:** Spins up standard virtual environments to build keys, test targets, and deliver web code straight to remote hosts

5. GitHub Encrypted Secrets

- **Definition:** A secure, encrypted storage workspace designed to save vital keys and authentication credentials inside a project.
- **Purpose:** Restricts visibility of crucial platform details to prevent structural account leaks.
- **Role:** Obfuscates private structural text strings such as EC2_KEY files from appearing within public logging records.

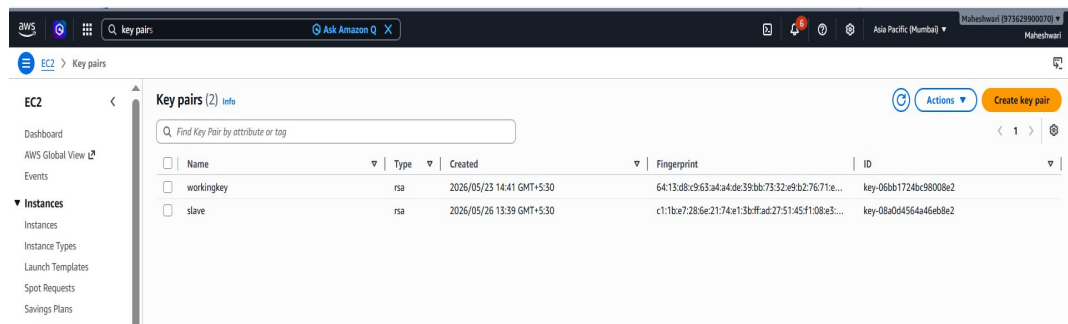
Architecture Overview



Implementation

Step 1: Create an EC2 key pair and download the PEM file securely.

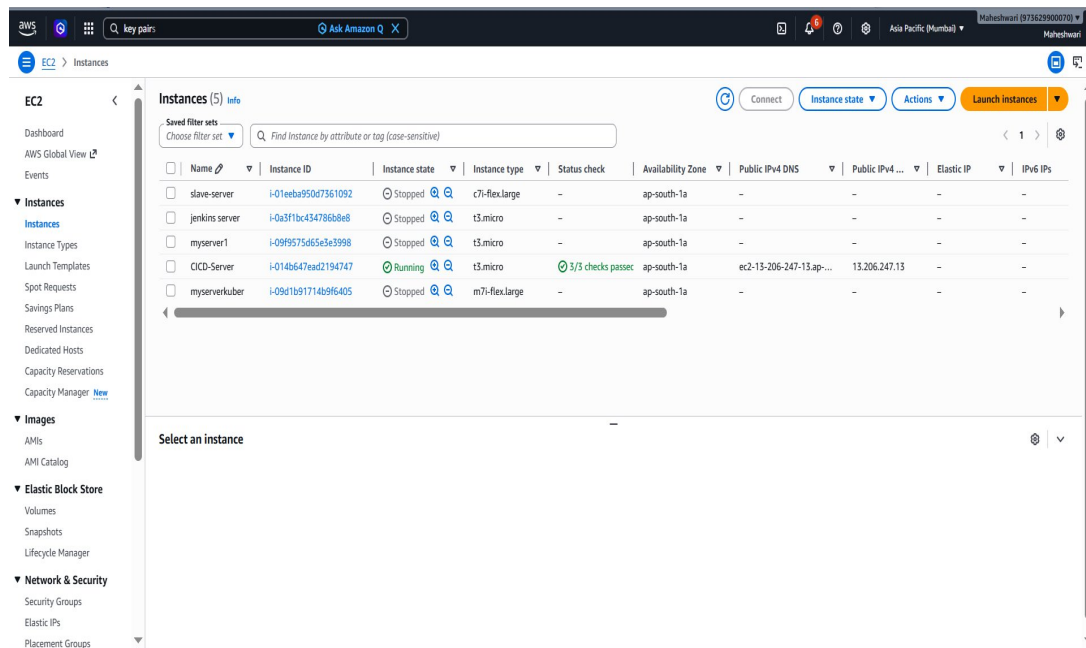
- Go to AWS Console → IAM (or EC2 Key Pairs dashboard).
- Generate a new RSA-based. pem private key pair.
- Download the private key file safely to your computer and record its name.
- Note: IAM tracks this key signature to ensure that only connections with the matching private key can access the EC2 compute resources.



Step 2: Launch an Amazon Linux 2023 EC2 instance with ports 22 and 80 open.

Go to AWS Console → EC2 → Launch Instances.

- Choose Amazon Linux 2023 as the base OS.
- Configure: Default VPC, Default Subnet, and set Auto-assign Public IP to Enable.
- Configure Security Group: Add Inbound Rule for SSH (Port 22) and HTTP (Port 80) sourced from anywhere (0.0.0.0/0).
- Under the Key Pair section, select the IAM/EC2 key pair created.
- Click Launch Instance and record its assigned Public IPv4 Address.



Step 3: Install and configure Apache web server using:

Install and activate the Apache web server package by running:

```
bash
```

```
sudo dnf update -y
```

```
sudo dnf install httpd -y
```

```
sudo systemctl enable httpd
```

```
sudo systemctl start httpd
```

```

AWS
Search [Alt+S]
Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

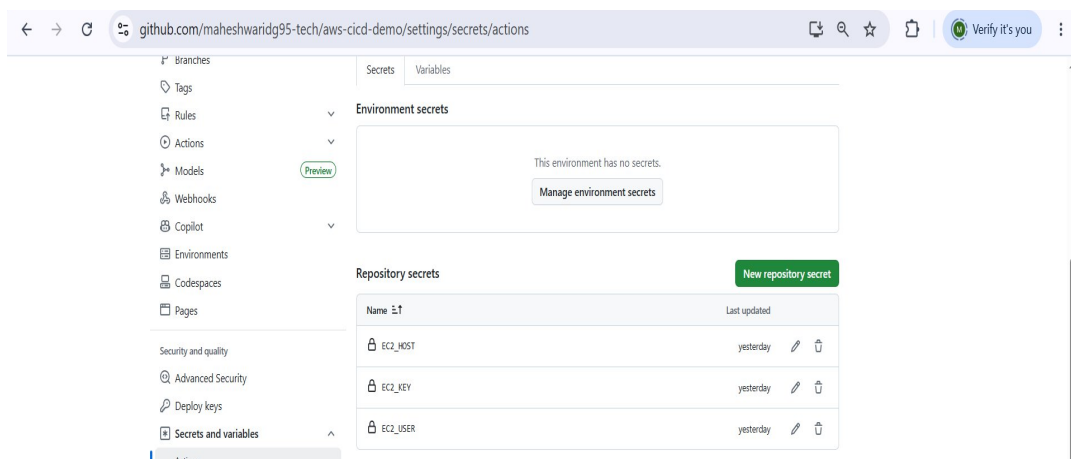
Last login: Mon Jun  8 07:32:56 2026 from 13.233.177.5
[ec2-user@ip-172-31-2-127 ~]$ sudo su
[root@ip-172-31-2-127 ec2-user]# sudo systemctl restart httpd
[root@ip-172-31-2-127 ec2-user]# sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-06-08 13:26:05 UTC; 11s ago
     Docs: man:httpd.service(8)
   Main PID: 122839 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 1014)
    Memory: 13.5M
       CPU: 73ms
   CGroup: /system.slice/httpd.service
           └─122839 /usr/sbin/httpd -DFOREGROUND
             └─122841 /usr/sbin/httpd -DFOREGROUND
               └─122842 /usr/sbin/httpd -DFOREGROUND
                 └─122843 /usr/sbin/httpd -DFOREGROUND
                   └─122844 /usr/sbin/httpd -DFOREGROUND

Jun 08 13:26:05 ip-172-31-2-127.ap-south-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jun 08 13:26:05 ip-172-31-2-127.ap-south-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jun 08 13:26:05 ip-172-31-2-127.ap-south-1.compute.internal httpd[122839]: Server configured, listening on: port 80
[root@ip-172-31-2-127 ec2-user]#
```

i-014b647ead2194747 (CICD-Server)
PublicIPs: 13.206.247.13 PrivateIPs: 172.31.2.127

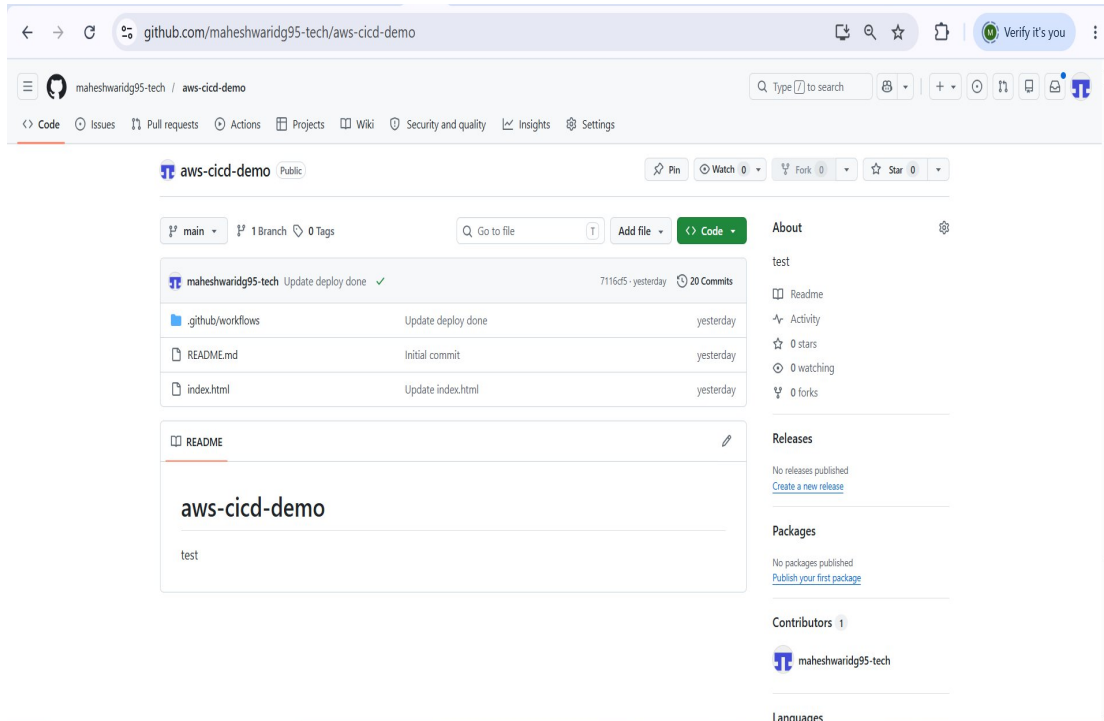
Step 4: Create a GitHub repository and upload index.html.

- Navigate to your GitHub repository dashboard in your web browser.
- Click Settings → expand Secrets and variables in the sidebar → click Actions.
- Under Repository secrets, click new repository secret.
- Add the following configuration item:
- Name: EC2_KEYValue: Open your downloaded. pem key file in a text editor, copy its entire content (including the BEGIN and END lines), and paste it here.
- EC2_HOST: Public IP address
- EC2_USER:ec2-user



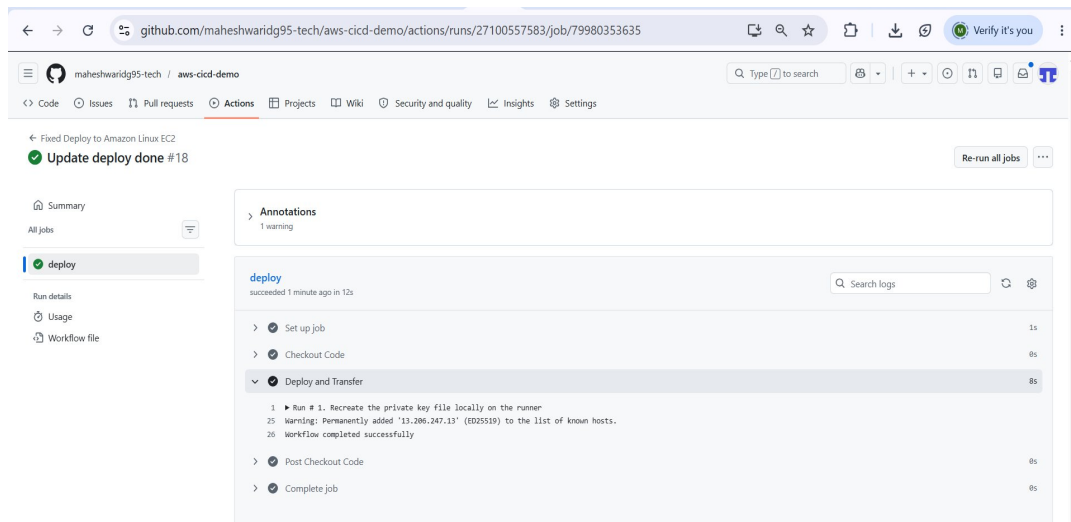
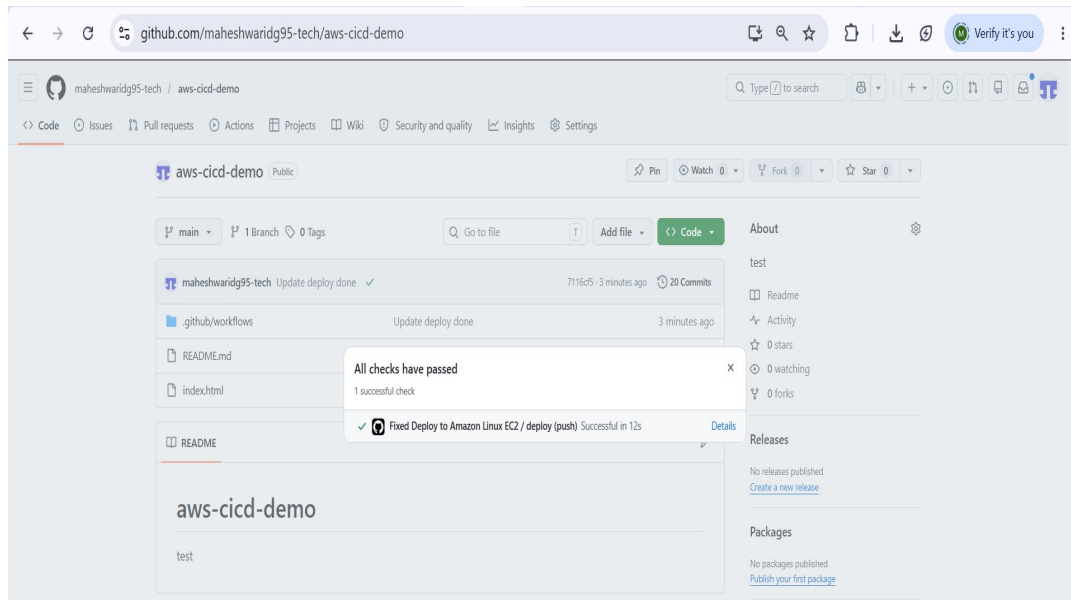
Step 5: Go back to the Code tab of your repository.

- Click Add file → Create new file.
- Name the path: .github/workflows/deploy.yml
- Testing Procedure
- Update index.html, commit changes, push to main branch, verify GitHub Actions success, and access the website through the EC2 public IP.



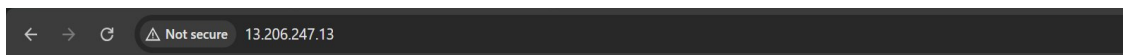
Step 6:

- Click on the Actions tab in your repository top menu.
- Click on the top active entry in the workflow execution logs list.
- Select the deploy job block to view the live execution tracking.
- Expand the Deploy and Transfer task segment to verify the execution.
- Confirm that the terminal displays: Workflow completed successfully
- Create .github/workflows/deploy.yml workflow file



Step 7:

- Test Live Application Accessibility
- Launch an isolated window inside your web browser application.
- Paste your target web address link into the navigation window: <http://13.206.247.13>.
- Verify that your updated repository code loads instantly on screen.



Deployment Successful!

This is my brand new version deployed automatically via GitHub Actions!

Trouble Shooting:

Issue 1: Deployment fails with empty targets (:/tmp/index.html: No such file or directory):

Cause: The pipeline engine is evaluating your host connection secrets as blank data strings, breaking command parsing.

Solution: Move non-sensitive connection metrics out of the complex repository configuration tab and insert them directly as local variables inside your script file (as handled in Step 5).

Issue 2: Terminal crashes with authorization flags (Permission denied (public key))

Cause: The system runner is sending invalid account user metadata or utilizing a damaged deployment authentication certificate string rejected by AWS identity filters.

Solution: Ensure your username string is manually forced to ec2-user in all lowercase formatting. If failures continue, erase your EC2_KEY entry from your settings repository panel and apply a fresh copy of your raw text key code file.

Conclusion:

The AWS CI/CD Web Deployment Pipeline provides an instantaneous and resilient method for pushing live code modifications from code tracking dashboards directly to internet servers. By combining GitHub Actions workflows with AWS IAM credential verification and automated file routing, the workflow removes manual operational steps, limits deployment human error, and controls overall management friction. This automated design pattern scales cleanly to handle expansive application directories, accommodates diverse multi-branch logic, and remains entirely functional within global free tier cloud accounts.